



MODULE

ARCHITECTURE D'API

Formation Professionnelle

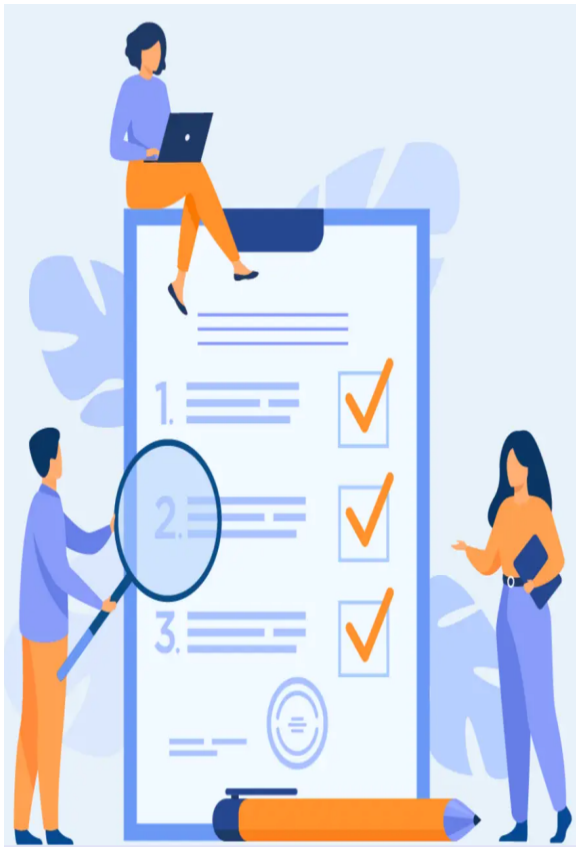
SOAP • REST • GraphQL

Enseignant : Serigne Diagne

778811181

sdiagne148@gmail.com

Plan du Module



⇒ Styles architecturaux

- SOAP
- REST
- GraphQL

⇒ Design d'API

- Ressources
- URIs
- Verbes
- Codes retour

⇒ Documentation & Standardisation

- OpenAPI / Swagger

⇒ Sécurité

- API Keys
- OAuth2
- OpenID Connect

⇒ Patterns Microservices

- Gateway
- Discovery
- Circuit Breaker

⇒ Performance

- Caching
- Rate Limiting
- Webhooks

Exigences d'un projet informatique

Exigences fonctionnelles

Une application est créée pour répondre, tout d'abord, aux besoins fonctionnels des entreprises.

Exigences Techniques

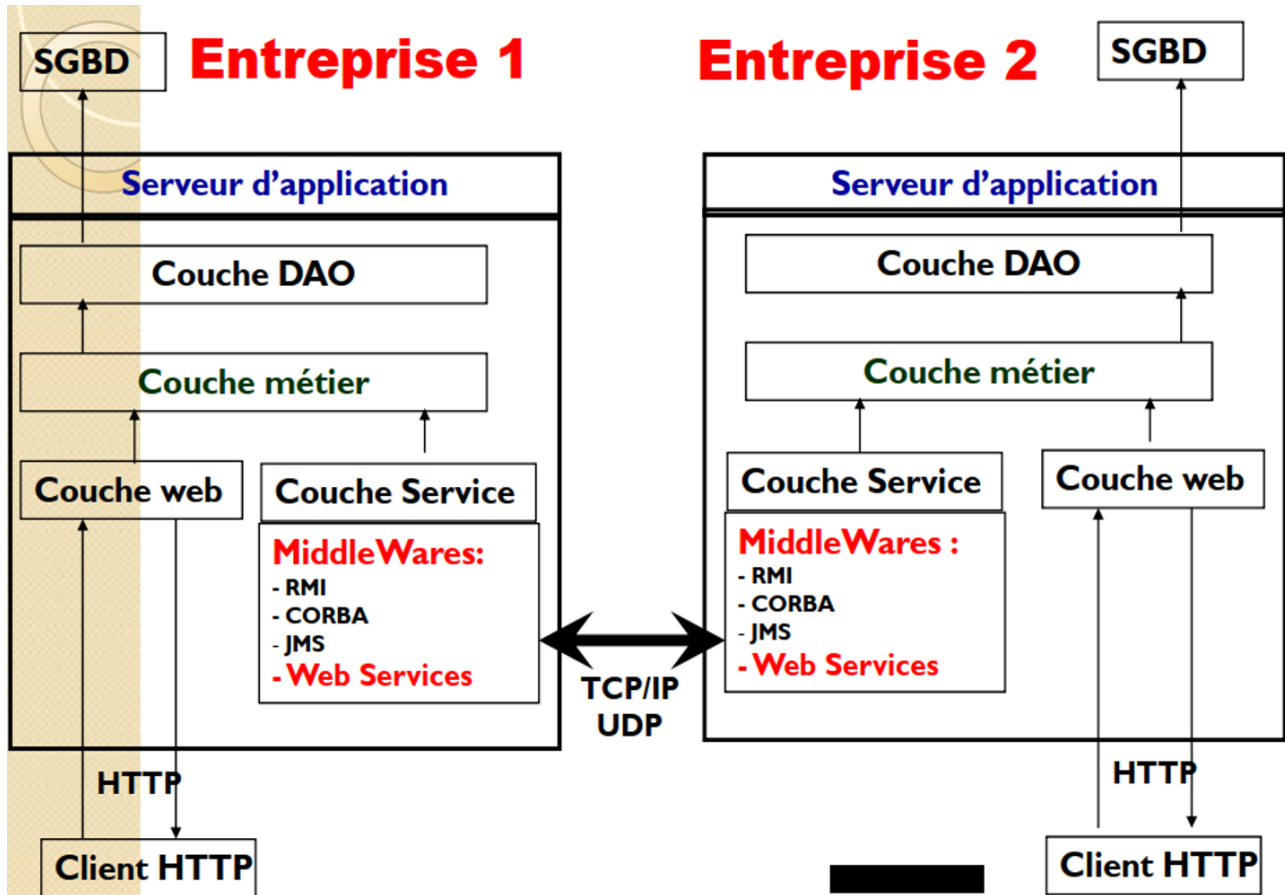
- Les performances :
 - ✓ Temps de réponse
 - ✓ Haute disponibilité et tolérance aux pannes
 - ✓ Éviter le problème de montée en charge
- La maintenance :
 - ✓ Une application doit évoluer dans le temps.
 - ✓ Doit être fermée à la modification et ouverte à l'extension
- Sécurité
- Portabilité
- Distribution
- Capacité de communiquer avec d'autres applications distantes.
- Capacité de fournir le service à différents types de clients (Desk TOP, Mobile, SMS, http...)
-
- Coût du logiciel

Constat

Il est très difficile de développer un système logiciel qui respecte ces exigences sans **utiliser l'expérience des autres** :

- Serveur d'application :
 - ✓ Apache Server Web
 - ✓ ...
- Framework pour l'Inversion de contrôle :
 - ✓ Spring (Conteneur léger)
- Frameworks :
 - ✓ Mapping objet relationnel (ORM): Entity Framework(.Net), Hibernate (Java), ...
 - ✓ Applications Web: Django, JSF, ASP .Net, SpringMVC
 - ✓
- Middlewares :
 - ✓ RMI, CORBA : Applications distribuées
 - ✓ JAXWS (java) / SPYNE(Django) pour Web services SOAP
 - ✓ JAXRS (java) / rest_framework (Django) pour les Web services RESTful
 - ✓ JMS : Communication asynchrone entre les applications

Vision globale d'une architecture distribuée



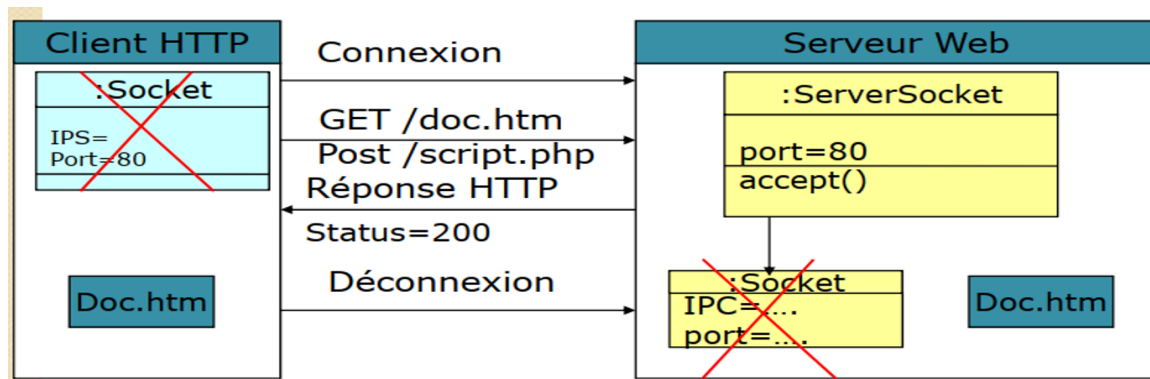
Introduction aux web services

Qu'est qu'un web service ?

- Les Web Services sont des composants web basés sur Internet (**HTTP**) qui exécutent des tâches précises et qui respectent un format spécifique (**XML/JSON**).
- Ils permettent aux applications de faire appel à des fonctionnalités à distance en simplifiant ainsi l'échange de données.
- Les Web Services permettent aux applications de dialoguer à travers le réseau, indépendamment de :
 - ✓ leur plate-forme d'exécution
 - ✓ et de leur langage d'implémentation.
- Ils s'inscrivent dans la continuité d'initiatives telles que
 - ✓ CORBA (*Common Object Request Broker Architecture*, de l'OMG) en apportant toutefois une réponse plus simple, s'appuyant sur des technologies et standards reconnus et maintenant acceptés de tous.

Le protocole Http

- **HTTP : HyperText Transfert Protocol**
 - ✓ Protocole qui permet au client de récupérer des documents du serveur
 - ✓ Ces documents peuvent être statiques (contenu qui ne change pas : HTML, PDF, Image, etc..) ou dynamiques (Contenu généré dynamiquement au moment de la requête : PHP, JSP, ASP, DJANGO...)
 - ✓ Ce protocole permet également de soumissionner les formulaires
- **Fonctionnement (très simple en HTTP/1.0)**
 - ✓ Le client se connecte au serveur (Créer une socket)
 - ✓ Le client demande au serveur un document : Requête HTTP
 - ✓ Le serveur renvoi au client le document (status = 200) ou d'une erreur
 - ✓ (status = 404 quand le document n'existe pas)
 - ✓ Déconnexion
- **Connexion**



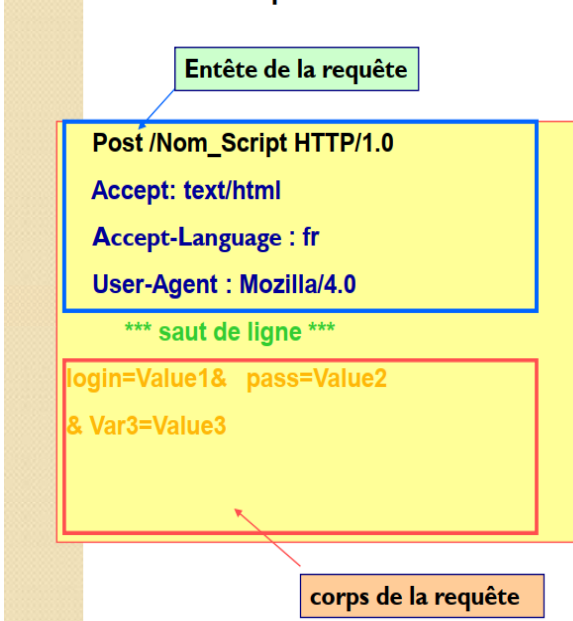
Méthodes du protocole Http

Une requête HTTP peut être envoyée en utilisant les méthodes suivantes :

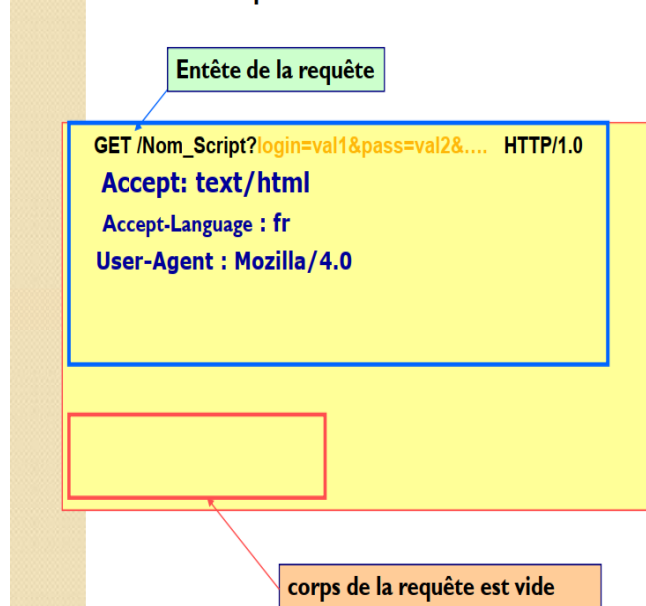
- **GET** : pour récupérer le contenu d'un document
- **POST** : pour soumissionner des formulaires (Envoyer, dans la requête, des données saisies par l'utilisateur)
- **PUT** : pour envoyer un fichier du client vers le serveur
- **DELETE** : permet de demander au serveur de supprimer un document.
- **HEAD** : permet de récupérer les informations sur un document (Type, Capacité, Date de dernière modification etc...)

Exemple de requêtes

Le client envoie la requête : Méthode POST



Le client envoie la requête : Méthode GET



Exemple de réponse :

Le Serveur retourne la réponse :



Styles Architecturaux : SOAP vs REST vs GraphQL

1.1 Qu'est-ce qu'une API ?

Une API (Application Programming Interface) est un contrat de communication entre deux systèmes logiciels. Elle définit comment les composants interagissent : quels messages peuvent être envoyés, sous quelle forme, et quelles réponses attendre.

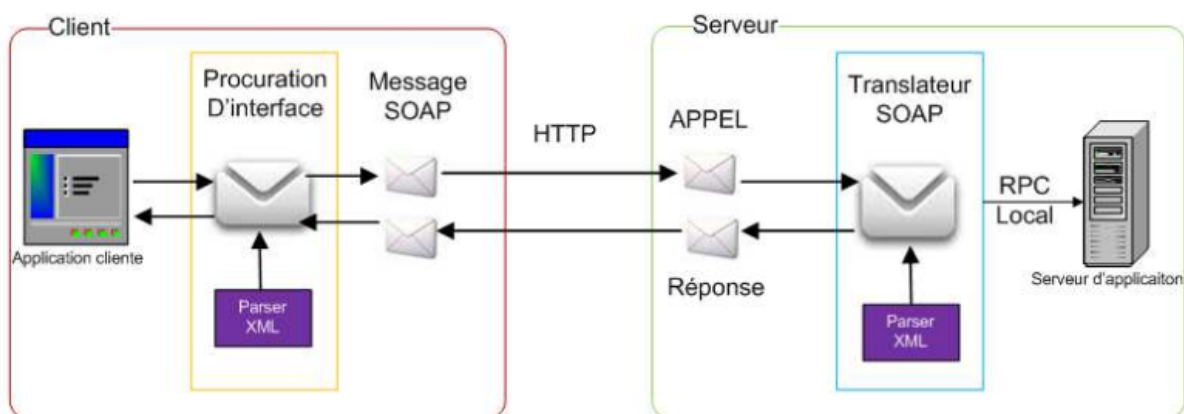
Analogie du Menu de Restaurant

- | |
|--|
| Le client (front-end) ne connaît pas la cuisine (back-end). |
| Le menu (API) liste ce qui est disponible et comment le commander. |
| Le serveur (HTTP) transporte les commandes et les plats. |
| → L'API est l'interface visible entre deux mondes distincts. |

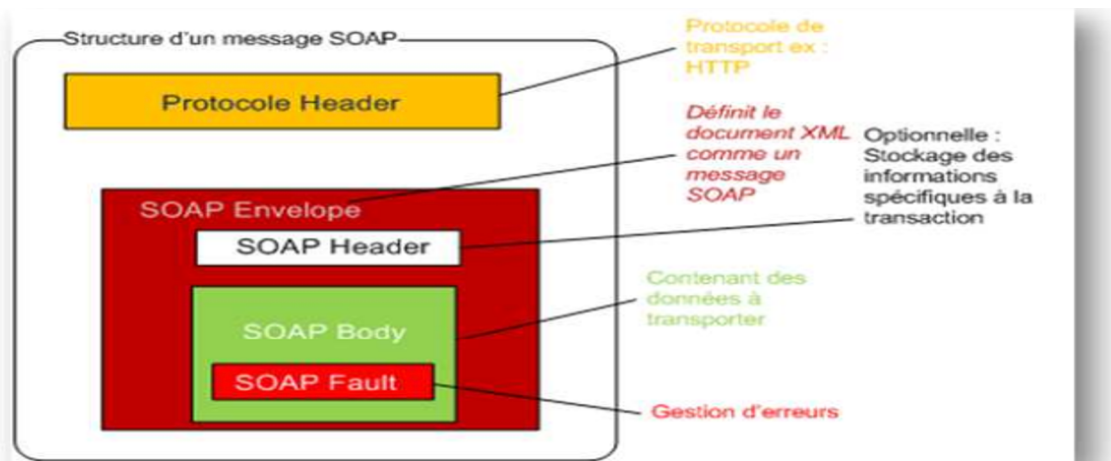
1.2 Le protocole SOAP

Définition et caractéristiques

- SOAP(Simple Object Access Protocol) est un protocole d'invocation de méthodes sur des services distants, **basé sur XML**,
- SOAP a pour principal objectif **d'assurer la communication entre machines**.
- Le protocole permet d'appeler une méthode **RPC** (Remote Procedure Call) et d'envoyer des messages aux machines distantes via un protocole de transport (**HTTP** ou **SMTP**).
- Contrat de service défini par un fichier **WSDL** (Web Services Description Language)
- Support natif des transactions, de la sécurité WS-Security et de la tolérance aux pannes
- Principalement utilisé dans les systèmes bancaires, télécoms, et ERP d'entreprise
- Ce protocole est très bien adapté à l'utilisation des services Web, car il permet de fournir au client une grande quantité d'informations récupérées sur un réseau de serveurs tiers, voyez :



Structure d'un message SOAP



```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding">
  <soap:Header>
    <!-- en-tête -->
  </soap:Header>
  <soap:Body>
    <!-- corps -->
  </soap:Body>
</soap:Envelope>
```

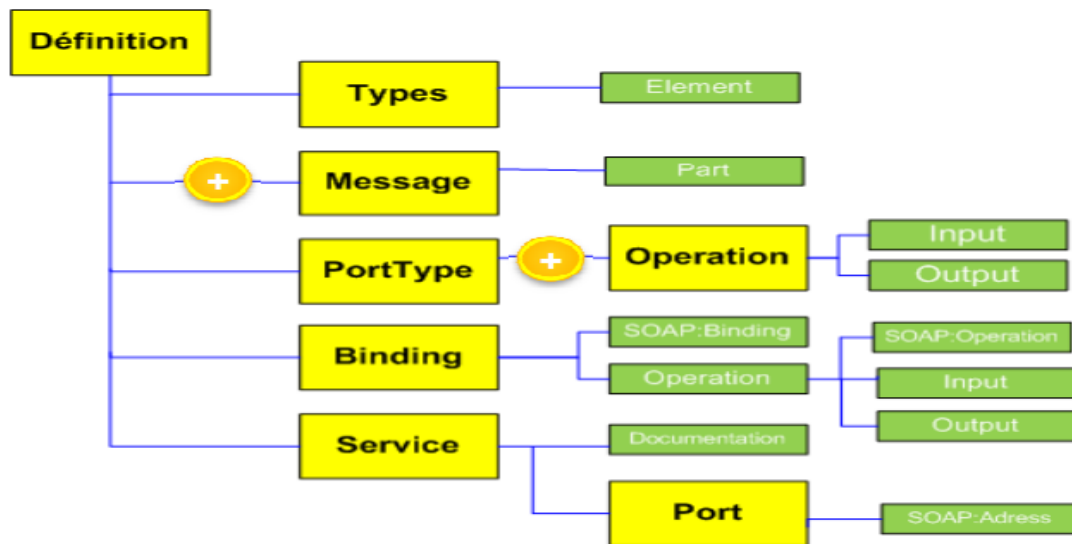
- **SOAP envelope** (enveloppe)
 - ✓ Est l'élément de base du message SOAP.
 - ✓ L'enveloppe contient la spécification des espaces de désignation (namespace) et du codage de données.
- **SOAP header** (entête)
 - ✓ est une partie facultative qui permet d'ajouter des fonctionnalités à un message SOAP de manière décentralisée sans agrément entre les parties qui communiquent.
 - ✓ L'entête est utile surtout, quand le message doit être traité par plusieurs intermédiaires.
- **SOAP body** (corps) est un *container* pour les informations mandataires à l'intention du récepteur du message, il contient les méthodes et les paramètres qui seront exécutés par le destinataire final.
- **SOAP fault** (erreur) est un élément facultatif défini dans le corps SOAP et qui est utilisé pour reporter les erreurs.

Exemple :

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <auth:Token>Bearer abc123</auth:Token>
  </soap:Header>
  <soap:Body>
    <GetBookRequest>
      <BookId>42</BookId>
    </GetBookRequest>
  </soap:Body>
</soap:Envelope>
```

Structure du WSDL

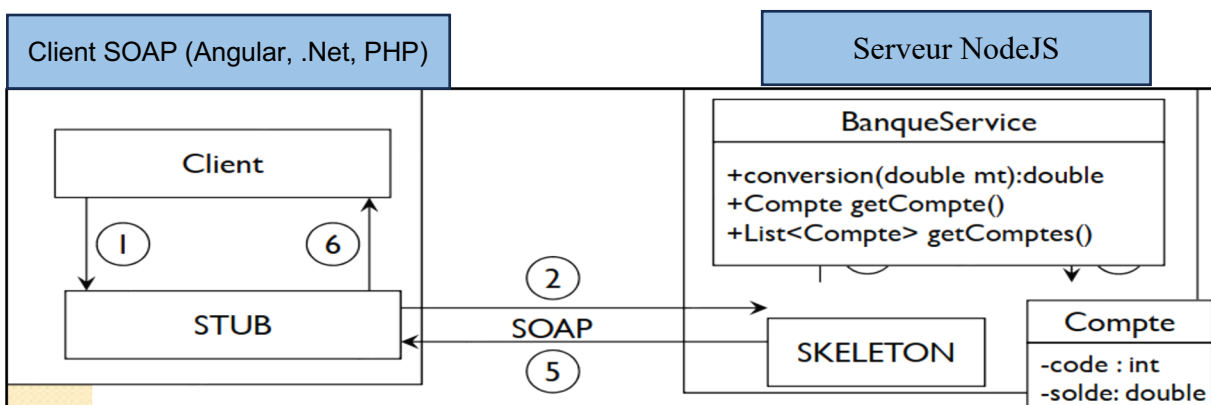
- Un document WSDL se compose d'un ensemble d'éléments décrivant les types de données utilisés par le service, les messages que le service peut recevoir, ainsi que les liaisons SOAP associées à chaque message.
- Le schéma suivant illustre la structure du langage WSDL qui est un document XML, en décrivant les relations entre les sections constituant un document WSDL.



Un fichier WSDL contient donc sept éléments.

- ✓ **Types** : fournit la définition de types de données utilisés pour décrire les messages échangés.
- ✓ **Messages** : représente une définition abstraite (noms et types) des données en cours de transmission.
- ✓ **PortTypes** : décrit un ensemble d'opérations. Chaque opération a zéro ou un message en entrée, zéro ou plusieurs messages de sortie ou d'erreurs.
- ✓ **Binding** : spécifie une liaison entre un <portType> et un protocole concret (SOAP, HTTP...).
- ✓ **Service** : indique les adresses de port de chaque liaison.
- ✓ **Port** : représente un point d'accès de services défini par une adresse réseau et une liaison.
- ✓ **Opération** : c'est la description d'une action exposée dans le port.

Architecture



1. Le client demande au stub de faire appel à la méthode conversion (12)
2. Le Stub se connecte au Skeleton et lui envoie une requête SOAP
3. Le Skeleton fait appel à la méthode du web service
4. Le web service retourne le résultat au Skeleton
5. Le Skeleton envoie le résultat dans une la réponse SOAP au Stub
6. Le Stub fournit le résultat au client

Mise en œuvre des web services SOAP avec Node.Js

- Créer un web service NodeJS en utilisant la bibliothèque **node-soap** qui permet de :
 - ✓ Convertir un montant de l'euro en DH
 - ✓ Consulter un compte sachant son id
 - ✓ Consulter une liste de comptes
- Tester le web service avec
 - ✓ Un analyseur SOAP (SoapUI ou Oxygen, etc...)
 - ✓ Créer un client Java
 - ✓ Créer un client PHP/ Laravel

Implémentation :

Étape 1 : Préparation du projet

Créez un dossier et installez les dépendances nécessaires :

```
mkdir tp-soap-node && cd tp-soap-node  
npm init -y  
npm install soap express
```

Étape 2 : Création du contrat WSDL (service.wsdl)

C'est la pièce maîtresse. Ce fichier décrit nos trois opérations : **convertirEuroDH**, **getCompte**, et **getComptes**.

```
<?xml version="1.0" encoding="UTF-8"?>  
<definitions name="BanqueService"  
  targetNamespace="http://exemple.org"  
  xmlns:tns="http://exemple.org"  
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"  
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"  
  xmlns="http://schemas.xmlsoap.org/wsdl/">  
  
  <!-- Schémas XSD : obligatoire pour listes / types composés (pas de tns:Compte[]) -->  
  <types>  
    <xsd:schema targetNamespace="http://exemple.org" xmlns:tns="http://exemple.org">  
      <xsd:complexType name="Compte">  
        <xsd:sequence>  
          <xsd:element name="id" type="xsd:int"/>  
          <xsd:element name="solde" type="xsd:double"/>  
        </xsd:sequence>  
      </xsd:complexType>  
    </xsd:schema>  
  </types>  
</definitions>
```

```

</xsd:complexType>
<xsd:complexType name="ListeComptes">
  <xsd:sequence>
    <xsd:element name="compte" type="tns:Compte" minOccurs="0" maxOccurs="unbounded"/>
  </xsd:sequence>
</xsd:complexType>
</xsd:schema>
</types>

<message name="ConvertirRequest"><part name="montant" type="xsd:double"/></message>
<message name="ConvertirResponse"><part name="resultat" type="xsd:double"/></message>
<message name="GetCompteRequest"><part name="id" type="xsd:int"/></message>
<message name="GetCompteResponse">
  <part name="id" type="xsd:int"/>
  <part name="solde" type="xsd:double"/>
</message>
<message name="GetComptesRequest"/>
<message name="GetComptesResponse">
  <part name="comptes" type="tns:ListeComptes"/>
</message>

<portType name="BanquePortType">
  <operation name="convertirEuroDH">
    <input message="tns:ConvertirRequest"/>
    <output message="tns:ConvertirResponse"/>
  </operation>
  <operation name="getCompte">
    <input message="tns:GetCompteRequest"/>
    <output message="tns:GetCompteResponse"/>
  </operation>
  <operation name="getComptes">
    <input message="tns:GetComptesRequest"/>
    <output message="tns:GetComptesResponse"/>
  </operation>
</portType>

<binding name="BanqueBinding" type="tns:BanquePortType">
  <soap:binding style="rpc" transport="http://schemas.xmlsoap.org/soap/http"/>
  <operation name="convertirEuroDH">
    <soap:operation soapAction="convertir"/><input><soap:body use="literal"/></input><output><soap:body use="literal"/></output>
  </operation>
  <operation name="getCompte">
    <soap:operation soapAction="getCompte"/><input><soap:body use="literal"/></input><output><soap:body
use="literal"/></output>
  </operation>
  <operation name="getComptes">
    <soap:operation soapAction="getComptes"/><input><soap:body use="literal"/></input><output><soap:body
use="literal"/></output>

```

```

    </operation>
  </binding>

  <service name="BanqueService">
    <port name="BanquePort" binding="tns:BanqueBinding">
      <soap:address location="http://localhost:8000/banque"/>
    </port>
  </service>
</definitions>

```

Étape 3 : Implémentation du Serveur (server.js)

Ici, nous définissons la logique métier pour traiter les requêtes XML.

```

const soap = require('soap');
const express = require('express');
const fs = require('fs');
const http = require('http');

const app = express();

// Simulation d'une base de données
const comptes = [
  { id: 1, solde: 5000 },
  { id: 2, solde: 12500 }
];

const serviceObject = {
  BanqueService: { // --- Doit matcher <service name="BanqueService"> dans service.wsdl
    BanquePort: { // --- Doit matcher <port name="BanquePort" binding="tns:BanqueBinding"> dans service.wsdl
      convertirEuroDH: function(args) {
        return { resultat: args.montant * 11 };
      },
      getCompte: function(args) {
        const compte = comptes.find(c => c.id == args.id);
        return compte || { id: 0, solde: 0 };
      },
      getComptes: function() {
        return { comptes: { compte: comptes } };
      }
    }
  }
};

const xml = fs.readFileSync('service.wsdl', 'utf8');

const server = http.createServer(app);

server.listen(8000, () => {
  soap.listen(server, '/banque', serviceObject, xml, (err) => {

```

```

    if (err) {
      console.error('Erreur lors du chargement du WSDL :', err);
      process.exit(1);
    }
    console.log('Service SOAP actif sur http://localhost:8000/banque?wsdl');
  });
});

```

Étape 4 : Test avec un Client JavaScript (client.js)

Le client va consommer le WSDL pour savoir comment appeler les fonctions.

```

const soap = require('soap');
const path = require('path');

const url = 'http://localhost:8000/banque?wsdl';
const wsdlPath = path.join(__dirname, 'service.wsdl');
const endpoint = 'http://localhost:8000/banque';

async function runClient() {
  try {
    //const client = await soap.createClientAsync(wsdlPath, { endpoint });
    const client = await soap.createClientAsync(url);

    // Appel de la méthode
    const [result] = await client.convertirEuroDHAsync({ montant: 100 });

    console.log('--- RÉSULTATS DU TEST ---');
    console.log('100€ en DH :', result.resultat);

    const [compte] = await client.getCompteAsync({ id: 2 });
    console.log('Infos Compte :', compte);

    // Entrée vide en style RPC : node-soap exige `null`, pas `{}`
    const [comptes] = await client.getComptesAsync(null);
    console.log('Comptes :', JSON.stringify(comptes, null, 2));

  } catch (err) {
    console.error("Erreur technique :", err.message);
  }
}

runClient();

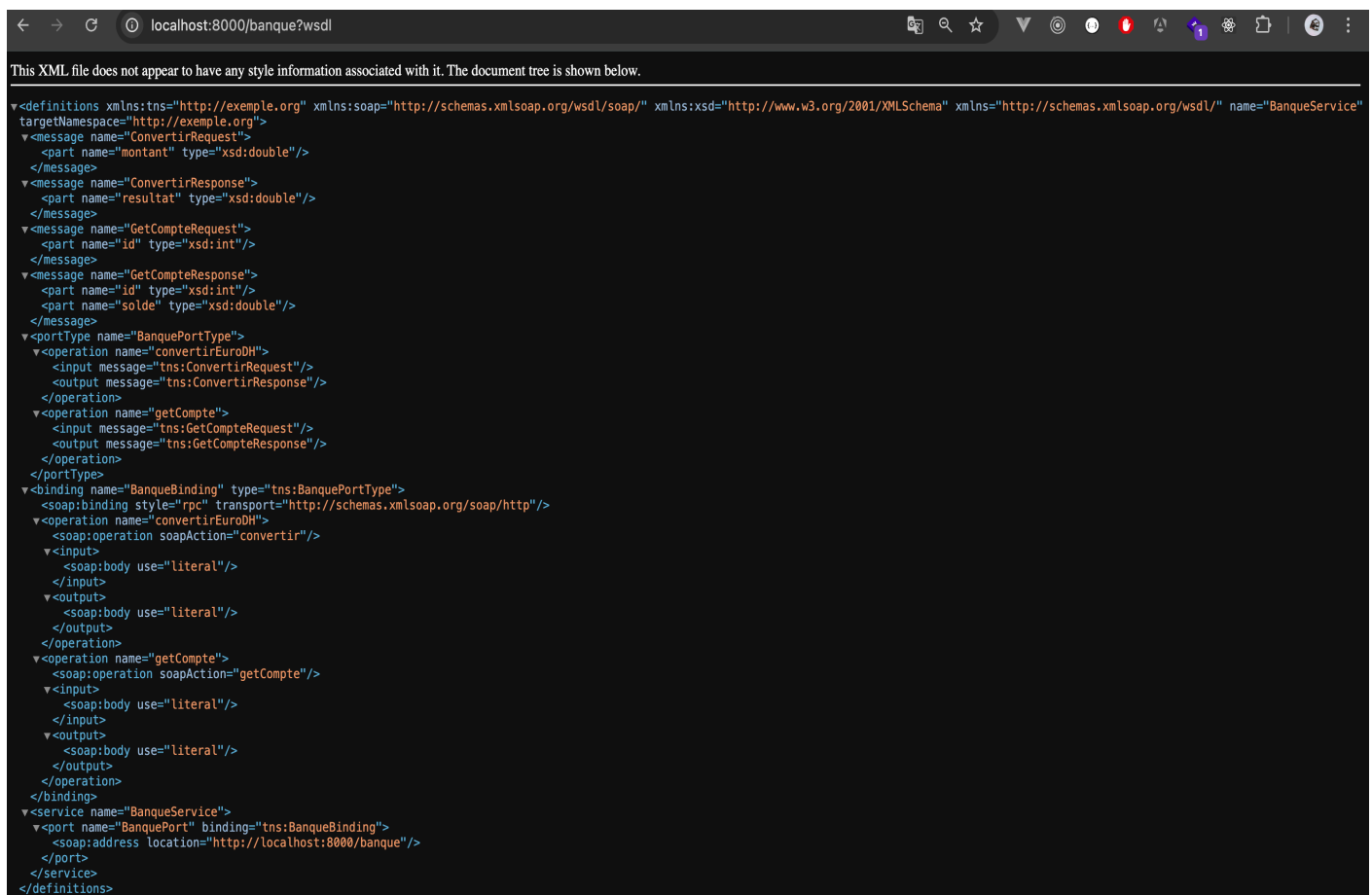
```

1. Lancer le serveur : `node server.js`
2. Tester : `node client.js` dans un autre terminal

```
● → tp-soap-node node client.js
--- RÉSULTATS DU TEST ---
100€ en DH : 1100
Infos Compte : { id: 2, solde: 12500 }
Comptes : {
  "comptes": {
    "compte": [
      {
        "id": "1",
        "solde": "5000"
      },
      {
        "id": "2",
        "solde": "12500"
      }
    ]
  }
}
```

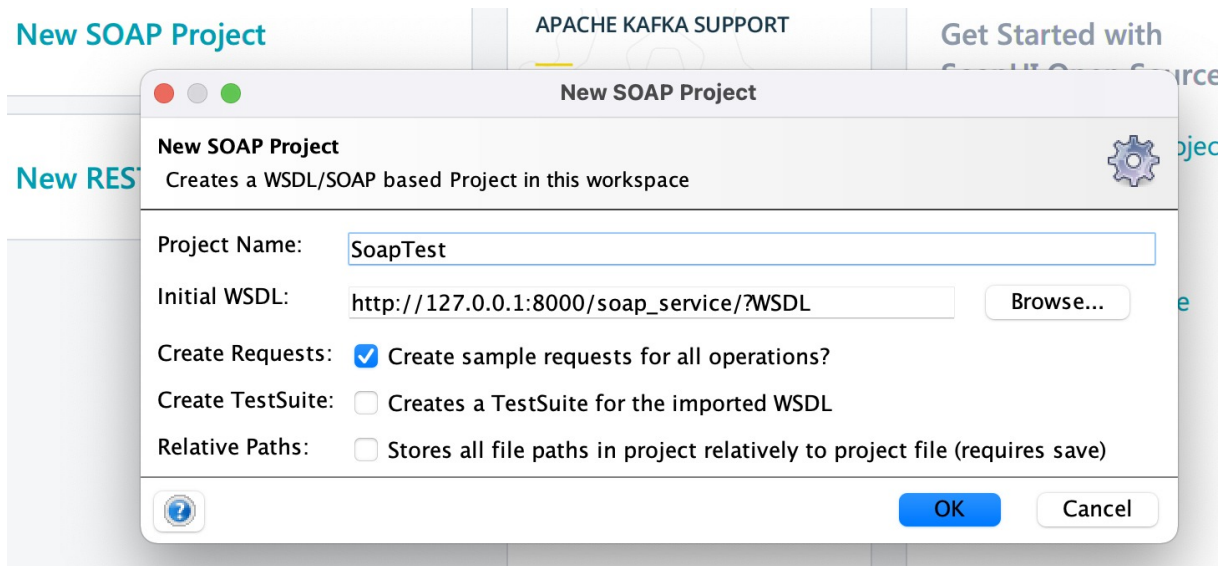
Pour Visualiser le WSDL, vous pouvez utiliser un navigateur web

<http://localhost:8000/banque/?wsdl>



```
<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns:tns="http://exemple.org" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.org/wsdl/" name="BanqueService" targetNamespace="http://exemple.org">
  <message name="ConvertirRequest">
    <part name="montant" type="xsd:double"/>
  </message>
  <message name="ConvertirResponse">
    <part name="resultat" type="xsd:double"/>
  </message>
  <message name="GetCompteRequest">
    <part name="id" type="xsd:int"/>
  </message>
  <message name="GetCompteResponse">
    <part name="id" type="xsd:int"/>
    <part name="solde" type="xsd:double"/>
  </message>
  <portType name="BanquePortType">
    <operation name="convertirEuroDH">
      <input message="tns:ConvertirRequest"/>
      <output message="tns:ConvertirResponse"/>
    </operation>
    <operation name="getCompte">
      <input message="tns:GetCompteRequest"/>
      <output message="tns:GetCompteResponse"/>
    </operation>
  </portType>
  <binding name="BanqueBinding" type="tns:BanquePortType">
    <soap:binding style="rpc" transport="http://schemas.xmlsoap.org/soap/http"/>
    <operation name="convertirEuroDH">
      <soap:operation soapAction="convertir"/>
      <input>
        <soap:body use="literal"/>
      </input>
      <output>
        <soap:body use="literal"/>
      </output>
    </operation>
    <operation name="getCompte">
      <soap:operation soapAction="getCompte"/>
      <input>
        <soap:body use="literal"/>
      </input>
      <output>
        <soap:body use="literal"/>
      </output>
    </operation>
  </binding>
  <service name="BanqueService">
    <port name="BanquePort" binding="tns:BanqueBinding">
      <soap:address location="http://localhost:8000/banque"/>
    </port>
  </service>
</definitions>
```

Tester les méthodes du web service avec un analyseur SOAP : SoapUI



Avantages et inconvénients de SOAP

Avantages	Inconvénients
Contrat formel (WSDL)	Verbosité du XML
Sécurité intégrée (WS-Security)	Courbe d'apprentissage élevée
Support des transactions ACID	Moins adapté aux apps web modernes
Interopérabilité inter-langages	Performances réduites vs REST

1.3 L'architecture REST

Définition et principes fondateurs

- **REST (REpresentational State Transfer) ou RESTful** est un style d'architecture pour les systèmes hypermédia distribués,
- Créé par Roy Fielding en 2000 dans le chapitre 5 de sa thèse de doctorat.
- **REST** est un style d'architecture permettant de construire des applications (Web, Intranet, Web Service).
- Il s'agit d'un ensemble de conventions et de bonnes pratiques à respecter et non d'une technologie à part entière.
- L'architecture REST utilise les spécifications originelles du **protocole HTTP**, plutôt que de réinventer une surcouche (comme le font SOAP ou XML-RPC par exemple).
- Ce n'est pas un protocole, mais un ensemble de contraintes architecturales.

Les 6 contraintes REST

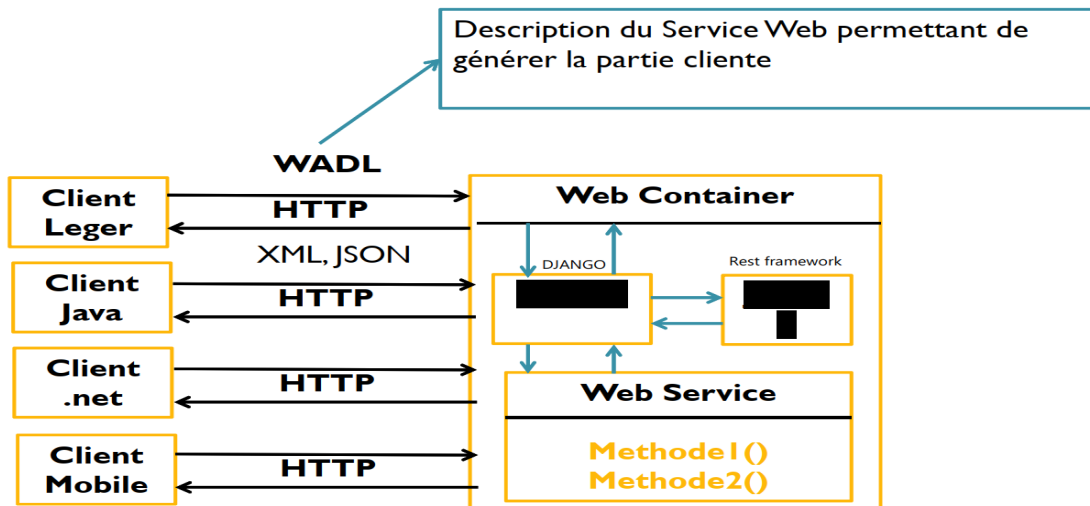
1. **Client-Serveur** : séparation claire des responsabilités
2. **Sans état (Stateless)** : chaque requête contient toutes les informations nécessaires
3. **Cache** : les réponses doivent indiquer si elles sont cachables

4. **Interface uniforme** : ressources identifiées par des URIs, représentations standardisées
5. **Système en couches** : le client ne sait pas s'il parle à un serveur final ou un intermédiaire
6. **Code à la demande (optionnel)** : le serveur peut envoyer du code exécutable

1.4 Les principes de base de REST & Design d'API

Ressources et représentations

En REST, tout est une ressource. Une ressource est un concept identifiable (un livre, un utilisateur, une commande). Elle est identifiée par un URI et peut avoir plusieurs représentations (JSON, XML, HTML).



Règles de RESTful

Règle n°1 :

- L'URI comme identifiant des ressources

Règle n°2 :

- Les verbes HTTP comme identifiant des opérations

Règle n°3 :

- Les réponses HTTP comme représentation des ressources

Règle n°4 :

- Les liens comme relation entre ressources

Règle n°5 :

- Un paramètre comme jeton d'authentification

Règle d'or REST
URI = Nom (ce qu'on manipule)
Verbe HTTP = Action (ce qu'on fait)
✓ GET /books/42 → Correct
X GET /getBook?id=42 → Incorrect (verbe dans l'URI)
X POST /deleteBook/42 → Incorrect (verbe dans l'URI + mauvais verbe HTTP)

Règle N°1 :

L'URI comme identifiant des ressources

- REST se base sur les **URI (Uniform Resource Identifier)** afin d'identifier une ressource.
- Ainsi une application se doit de construire ses URI (et donc ses URL) de manière précise, en tenant compte des **contraintes REST**.
- Il est nécessaire de prendre en compte la hiérarchie des ressources et la sémantique des URL pour les éditer :

Règles de nommage

- Utiliser des noms au **pluriel** pour les collections : /books, /users, /loans
- Noms en minuscules, **séparés par des tirets** : /book-categories
- Jamais de **verbes** dans les URIs
- **Hiérarchie** logique reflétant les relations : /users/{id}/loans
- **Versioning** dans l'URI : /api/v1/books

Exemples de construction d'URIs avec RESTful pour une bibliothèque

- **Liste des livres**
NOK : <http://mywebsite.com/book>
OK : <http://mywebsite.com/books>
- **Filtre et tri sur les livres**
NOK : <http://mywebsite.com/books/filtre/policier/tri/asc>
OK : <http://mywebsite.com/books?fitre=policier&tri=asc>
- **Affichage d'un livre**
NOK : <http://mywebsite.com/book/display/87>
OK : <http://mywebsite.com/books/87>
- **Tous les commentaires sur un livre**
NOK : <http://mywebsite.com/books/comments/87>
OK : <http://mywebsite.com/books/87/comments>
- **Affichage d'un commentaire sur un livre**
NOK : <http://mywebsite.com/books/comments/87/1568>
OK : <http://mywebsite.com/books/87/comments/1568>
- En construisant correctement les URI, il est possible de les trier, de les hiérarchiser et donc d'améliorer la compréhension du système.
- L'URL suivante peut alors être décomposée logiquement
 - ✓ <http://mywebsite.com/books/87/comments/1568> => **un commentaire pour un livre**
 - ✓ <http://mywebsite.com/books/87/comments> => **tous les commentaires pour un livre**
 - ✓ <http://mywebsite.com/books/87> => **un livre**
 - ✓ <http://mywebsite.com/books> => **tous les livres**

URI	Description
-----	-------------

GET /api/v1/books	Liste de tous les livres
GET /api/v1/books/{id}	Détails d'un livre spécifique
POST /api/v1/books	Créer un nouveau livre
PUT /api/v1/books/{id}	Remplacer un livre entier
PATCH /api/v1/books/{id}	Modifier partiellement un livre
DELETE /api/v1/books/{id}	Supprimer un livre
GET /api/v1/users/{id}/loans	Emprunts d'un utilisateur
POST /api/v1/books/{id}/borrow	Emprunter un livre (action)

Règle n°2 :

Les verbes HTTP comme identifiant des opérations

- La seconde règle d'une architecture REST est d'utiliser les verbes HTTP existants plutôt que d'inclure l'opération dans l'URI de la ressource.
- Ainsi, généralement pour une ressource, il y a 4 opérations possibles (**CRUD**) :
 - ✓ Créer (create)
 - ✓ Afficher (read)
 - ✓ Mettre à jour (update)
 - ✓ Supprimer (delete)
- HTTP propose les verbes correspondants :

CRUD et correspondances HTTP

Verbe HTTP	Opération CRUD	Idempotent ?	Corps ?
GET	Read (lecture)	Oui	Non
POST	Create (création)	Non	Oui
PUT	Update (remplacement)	Oui	Oui
PATCH	Update (modification partielle)	Non*	Oui
DELETE	Delete (suppression)	Oui	Non
HEAD	Métadonnées seulement	Oui	Non
OPTIONS	Capacités du serveur	Oui	Non

Idempotence en pratique
Une opération est idempotente si l'appeler plusieurs fois produit le même résultat.
GET /books/42 → toujours le même livre (idempotent)
DELETE /books/42 → 1ère fois: 204, 2ème fois: 404 (résultat stable: livre supprimé)
POST /books → crée un nouveau livre à chaque appel (NON idempotent)

PUT vs PATCH

```
# PUT - Remplacement complet (envoyer TOUS les champs)
PUT /api/v1/books/42
{
  "titre": "Clean Code",
  "auteur": "Robert C. Martin",
  "isbn": "978-0-13-235088-4",
  "disponible": false
}

# PATCH - Modification partielle (seulement les champs modifiés)
PATCH /api/v1/books/42
{
  "disponible": false
}
```

Exemple d'URL pour une ressource donnée (un livre par exemple) :

- **Créer un livre**
NOK : GET <http://mywebsite.com/books/create>
OK : POST <http://mywebsite.com/books>
- **Afficher**
NOK : GET <http://mywebsite.com/books/display/87>
OK : GET <http://mywebsite.com/books/87>
- **Mettre à jour**
NOK : POST <http://mywebsite.com/books/editor/87>
OK : PUT <http://mywebsite.com/books/87>
- **Supprimer**
NOK : GET <http://mywebsite.com/books/87/delete>
OK : DELETE <http://mywebsite.com/books/87>

Règle n°3 :

Les réponses HTTP comme représentation des ressources

- Il est important d'avoir à l'esprit que la réponse envoyée n'est pas une ressource, c'est la représentation d'une ressource.
- Ainsi, une ressource peut avoir plusieurs représentations dans des formats divers : **HTML, XML, CSV, JSON, etc.**
- C'est au client de définir quel format de réponse s'il souhaite recevoir via l'entête **Accept**.
- Il est possible de définir plusieurs formats.
 - ✓ **Réponse en HTML**
[GET /books](#)
[Host : mywebsite.com](#)
[Accept : text/html](#)
 - ✓ **Réponse en XML**

GET /books

Host : mywebsite.com

Accept: application/xml

Règle n°4 :

Les liens comme relation entre ressources

- Les liens d'une ressource vers une autre ont tous une chose en commun : ils indiquent la présence d'une relation.
- Il est cependant possible de la décrire afin d'améliorer la compréhension du système.
- Pour expliciter cette description et indiquer la nature de la relation, l'attribut **rel** doit être spécifier sur tous les liens.
- Ainsi l'IANA donne une liste de relation parmi lesquelles :
 - ✓ contents
 - ✓ edit
 - ✓ next
 - ✓ last
 - ✓ payment
 - ✓ etc.
- La liste complète sur le site de l'IANA :

<http://www.iana.org/assignments/link-relations/link-relations.xml>

Exemple de réponse en XML d'une liste paginée de livres :

```
<?xml>
<search>
  <link rel="self" title="self" href="http://mywebsite.com/books?q=policier&page=1&c=5" />
  <link rel="next" title="next" href="http://mywebsite.com/books?q=policier&page=2&c=5" />
  <link rel="last" title="last" href="http://mywebsite.com/books?q=policier&page=4&c=5" />
  <book>
    //...
  </book>
</search>
```

Règle n°5 :

Un paramètre comme jeton d'authentification

- C'est un des sujets les plus souvent abordé quand on parle de REST : comment authentifier une requête ?
- La réponse est très simple et est massivement utilisée par des APIs renommées (Google, Yahoo, etc.) : le **jeton d'authentification**.
- Chaque requête est envoyée avec un jeton (token) passé en paramètre **\$_GET** de la requête.
- Ce jeton temporaire est obtenu en envoyant une première requête d'authentification puis en le combinant avec nos requêtes.
- Ainsi, on peut construire le scénario suivant :

1. demande d'authentification

GET /users/123/authenticate?pass=lkdnsdf54d47894f5123002fds2sd360s0

```
<?xml>
<user>
  <id>123</id>
  <name>Nicolas Hachet</name>
</user>
<token>
  fsd531gfd5g5df31fdg3g3df45
</token>
```

2. accès aux ressources :

Cet token est ensuite utilisé pour générer un hash de la requête de cette façon :

```
hash = SHA1(token + requete)
hash = SHA1(fsd531gfd5g5df31fdg3g3df45 + "GET /books")
hash = 456894ds4q15sdq156sd1qsd1qsd156156
```

C'est cet hash qui est passé comme jeton afin de valider l'authentification pour cette requête :

```
GET /books?user=123&hash=456894ds4q15sdq156sd1qsd1qsd156156
```

1.5 JSON

- **JSON** (JavaScript Object Notation – Notation Objet issue de JavaScript) est un format léger d'échange de données.
- Il est facile à lire ou à écrire pour des humains. Il est aisément analysable ou général par des machines.
- JSON est un format texte complètement indépendant de tout langage, mais les conventions qu'il utilise seront familières à tout programmeur habitué aux langages descendant du C, comme par exemple : C lui-même, C++, C#, Java, JavaScript, Perl, Python et bien d'autres.
- Ces propriétés font de JSON un langage d'échange de données idéal.

Exemple XML/JSON

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<comptes>
  <compte type="courant">
    <code>1</code>
    <solde>4300</solde>
    <dateCreation>2012-11-11</dateCreation>
  </compte>
  <compte type="epargne">
    <code>2</code>
    <solde>96000</solde>
    <dateCreation>2012-12-11</dateCreation>
  </compte>
</comptes>
```

JSON

```
[
  {type:"courant" , code:1, solde:4300.50,dateCreation:"2012-11-11"},
  {type:"epargne" , code:1, solde:4300.00,dateCreation:"2012-11-11"}
]
```

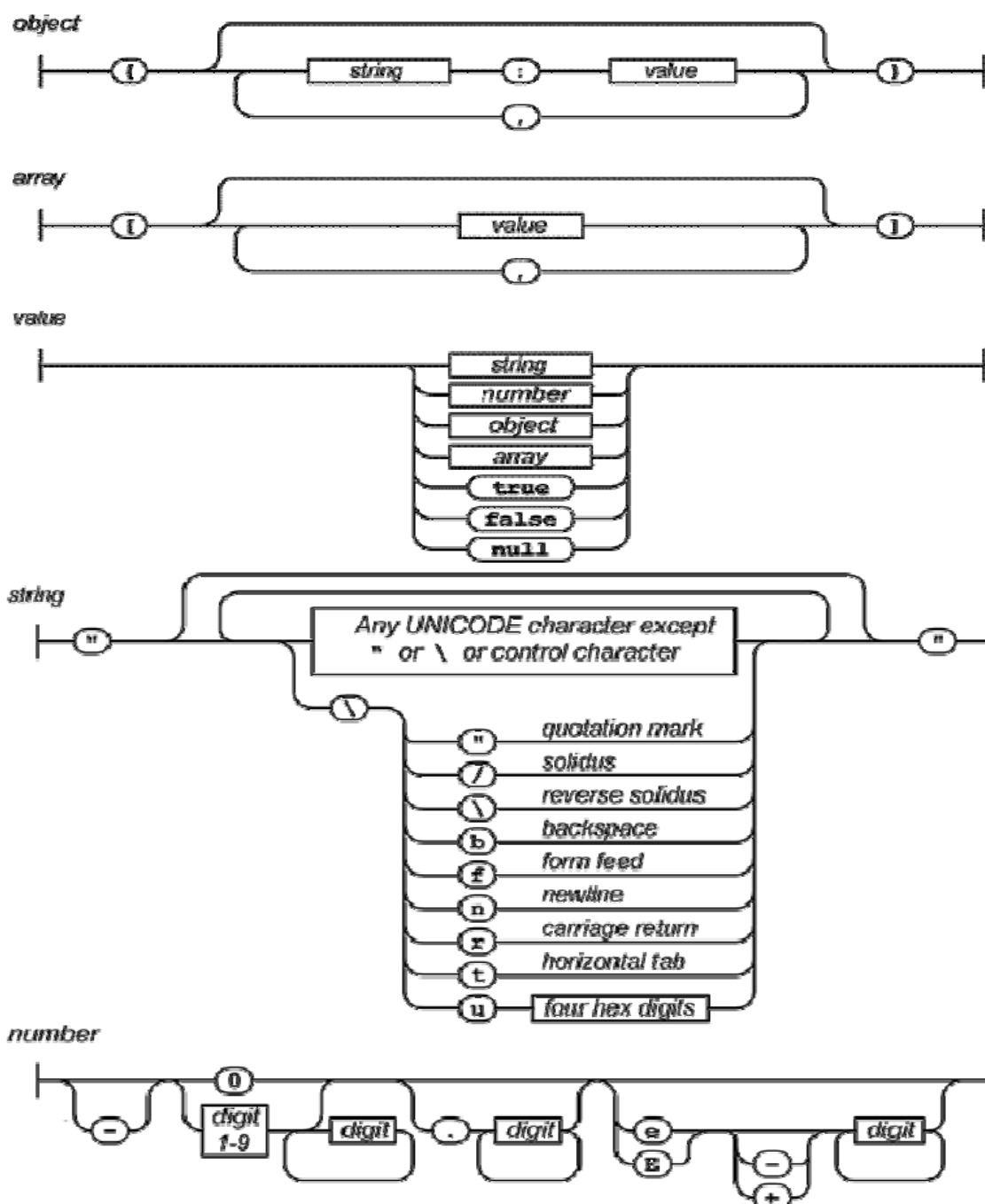
Structures de JSON

- JSON se base sur deux structures :
 - ✓ Une collection de couples **nom/valeur**.
 - ✓ Divers langages la réifient par un *objet*, un enregistrement, une structure, un dictionnaire, une table de hachage, une liste typée ou un tableau associatif.
- Ces structures de données sont universelles. Pratiquement tous les langages de programmation modernes les proposent sous une forme ou une autre.

Les structures de données JSON

En JSON, les structures de données prennent les formes suivantes :

- ✓ Un **objet**, qui est un ensemble de couples nom/valeur non ordonnés. Un objet commence par {(accolade gauche) et se termine par} (accolade droite). Chaque nom est suivi de : (deux points) et les couples nom/valeur sont séparés par, (virgule).
- ✓ Un **tableau** est une collection de valeurs ordonnées. Un tableau commence par [(crochet gauche) et se termine par] (crochet droit). Les valeurs sont séparées par, (virgule).
- ✓ Une **valeur** peut être soit une *chaîne de caractères* entre guillemets, soit un *nombre*, soit *true* ou *false* ou *null*, soit un *objet* soit un *tableau*. Ces structures peuvent être imbriquées.
- ✓ Une **chaîne de caractères** est une suite de zéro ou plus caractères Unicode, entre guillemets, et utilisant les échappements avec antislash. Un caractère est représenté par une chaîne d'un seul caractère.



Exemple REST – Récupérer un livre

GET /api/v1/books/42 HTTP/1.1

Host: api.bibliotheque.fr

Accept: application/json

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "id": 42,
  "titre": "Clean Code",
  "auteur": "Robert C. Martin",
  "disponible": true
}
```

1.6 Les Codes de Retour HTTP

Gestion du contenu : statut des réponses

- Lors de l'envoi de la réponse au client un code statut est retourné.
- **Réponse sans erreur** : Les statuts des réponses sans erreur s'échelonnent de 200 à 399
 - Le code est 200 « **OK** » pour les services retournant un contenu non vide
 - Le code est 204 « **No Content** » pour les services retournant un contenu vide
- **Réponse avec erreur** : Les statuts des réponses avec erreur s'échelonnent de 400 à 599
 - Une ressource non trouvée, le code de retour est 404 « **Not Found** »
 - Un type MIME en retour non supporté, 406 « **Not Acceptable** »
 - Une méthode HTTP non supportée, 405 « **Method Not Allowed** »

Familles de codes

Famille	Signification	Exemples
1xx	Informationnel	100 Continue
2xx	Succès	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved, 304 Not Modified
4xx	Erreur client	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Erreur serveur	500 Internal Error, 503 Service Unavailable

HTTP Status Codes	
Information [100 - 199] 100 - Continue 101 - Switching Protocols 102 - Processing 103 - Early hints	Success [200 - 299] 200 - Ok 201 - Created 202 - Accepted 204 - No Content 206 - Partial Content
Redirect [300 - 399] 300 - Multiple choices 301 - Moved Permanently 304 - Not Modified 307 - Temporary redirect 308 - Permanent redirect	Client Error [400 - 499] 400 - Bad request 401 - Unauthorized 403 - Forbidden 404 - Not found 409 - Conflict
Server Error [500 - 599] 500 - Internal server error 501 - Not implemented 502 - Bad gateway 503 - Service unavailable 504 - Gateway timeout	

Codes essentiels à maîtriser

Code	Nom	Usage typique
200	OK	GET/PUT/PATCH réussi avec corps de réponse
201	Created	POST réussi, ressource créée (+ header Location)
204	No Content	DELETE réussi, pas de corps de réponse
400	Bad Request	Corps de requête invalide ou paramètre manquant

401	Unauthorized	Token absent ou invalide (non authentifié)
403	Forbidden	Authentifié mais sans permission (non autorisé)
404	Not Found	Ressource introuvable
409	Conflict	Conflit d'état (ex: ISBN déjà existant)
422	Unprocessable Entity	Validation métier échouée
429	Too Many Requests	Rate limit dépassé
500	Internal Server Error	Erreur non gérée côté serveur

Exemple complet de réponse d'erreur

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{
  "type": "https://api.bibliotheque.fr/errors/validation",
  "title": "Validation Error",
  "status": 422,
  "detail": "Les données fournies ne sont pas valides.",
  "errors": [
    { "field": "isbn", "message": "Format ISBN-13 invalide" },
    { "field": "titre", "message": "Le titre est obligatoire" }
  ]
}
```

1.7 Bonnes pratiques de design d'API

Pagination

```
# Pagination par curseur (recommandée pour grands datasets)
GET /api/v1/books?page=2&limit=20&sort=titre&order=asc

# Réponse avec métadonnées de pagination
{
  "data": [...],
  "pagination": {
    "page": 2,
    "limit": 20,
    "total": 350,
    "totalPages": 18,
    "links": {
      "self": "/books?page=2&limit=20",
      "next": "/books?page=3&limit=20",
      "prev": "/books?page=1&limit=20"
    }
  }
}
```

Filtrage et recherche

```
# Filtres via query parameters
GET /api/v1/books?disponible=true&genre=roman&auteur=Zola

# Recherche full-text
GET /api/v1/books?q=javascript&fields=titre,description
```

Exemple de web service RESTful

- Consulter un compte
- Consulter la liste des comptes
- Effectuer un versement
- Effectuer un retrait
- Effectuer un virement
- Supprimer un compte

Implémentation :

???

1.8 GraphQL

Genèse et philosophie

GraphQL a été développé par Facebook en 2012 et open-sourcé en 2015. Il répond aux limitations de REST (over-fetching et under-fetching) en permettant au client de demander exactement les données dont il a besoin.

- Un **seul endpoint** : POST /graphql
- Le client définit la forme exacte de la réponse
- Support natif des mutations (modifications) et subscriptions (temps réel)
- Système de types fort avec un schéma auto-documenté

Exemple de requête GraphQL

```
# Requête GraphQL – Client demande exactement ce dont il a besoin
query {
  livre(id: 42) {
    titre
    auteur {
      nom
      nationalite
    }
    disponible
  }
}

# Réponse précise – aucun champ superflu
```

```

{
  "data": {
    "livre": {
      "titre": "Clean Code",
      "auteur": { "nom": "Robert C. Martin", "nationalite": "américaine" },
      "disponible": true
    }
  }
}

```

1.9 Tableau comparatif SOAP / REST / GraphQL

Critère	SOAP	REST	GraphQL
Format	XML	JSON / XML / YAML	JSON
Endpoints	Un par opération	Plusieurs (par ressource)	Un seul
Contrat	WSDL	OpenAPI (optionnel)	Schema SDL
Sécurité	WS-Security	OAuth2 / JWT	OAuth2 / JWT
Cache	Difficile	HTTP Cache natif	Complexe
Courbe	Élevée	Modérée	Modérée
Idéal pour	Systèmes d'entreprise	APIs publiques/web	Apps mobiles/BFF

Documentation et Standardisation : OpenAPI / Swagger

3.1 Pourquoi documenter une API ?

- Contrat entre équipes front-end et back-end (API-first design)
- Réduction du temps d'intégration pour les consommateurs tiers
- Génération automatique de code client (SDK)
- Tests automatisés basés sur la spécification
- Portail développeur professionnel

3.2 La spécification OpenAPI (OAS)

Historique

Swagger a été créé par Wordnik en 2011. En 2016, la spécification a été donnée à la Linux Foundation sous le nom OpenAPI Specification (OAS). La version actuelle est OAS 3.1.

Structure d'un fichier OpenAPI 3.1

```
openapi: 3.1.0
info:
  title: API Bibliothèque
  version: 1.0.0
  description: API REST pour la gestion d'une bibliothèque
  contact:
    name: Équipe API
    email: api@bibliotheque.fr

servers:
  - url: https://api.bibliotheque.fr/v1
    description: Serveur de production
  - url: http://localhost:3000/v1
    description: Serveur de développement

paths:
  /books:
    get:
      summary: Lister tous les livres
      operationId: listBooks
      tags: [Books]
      parameters:
        - name: page
          in: query
          schema: { type: integer, default: 1 }
        - name: limit
          in: query
          schema: { type: integer, default: 20, maximum: 100 }
      responses:
        '200':
```

```
    description: Liste des livres paginée
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/BookList'
  '401':
    $ref: '#/components/responses/Unauthorized'
```

Définir des schémas réutilisables

```
components:
  schemas:
    Book:
      type: object
      required: [titre, auteur, isbn]
      properties:
        id:
          type: integer
          readOnly: true
          example: 42
        titre:
          type: string
          minLength: 1
          maxLength: 200
          example: Clean Code
        auteur:
          type: string
          example: Robert C. Martin
        isbn:
          type: string
          pattern: '^[0-9]{13}$'
          example: '9780132350884'
        disponible:
          type: boolean
          default: true
        createdAt:
          type: string
          format: date-time
          readOnly: true
```

Documenter la sécurité

```
components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
    ApiKeyAuth:
      type: apiKey
```

```

    in: header
    name: X-API-Key

# Sécurité globale (tous les endpoints)
security:
  - BearerAuth: []

# Surcharge sur un endpoint spécifique (public)
paths:
  /public/books:
    get:
      security: [] # Pas d'auth requise

```

3.3 Swagger UI et outillage

Outil	Description
Swagger UI	Interface web interactive pour explorer et tester l'API
Swagger Editor	Éditeur en ligne avec validation en temps réel
Redoc	Documentation API statique élégante
Postman	Import OpenAPI + tests automatisés
openapi-generator	Génération de SDK client dans 50+ langages
Prism (Stoplight)	Mock server basé sur la spécification OpenAPI

3.4 Approche Design-First vs Code-First

Critère	Design-First	Code-First
Départ	Écrire la spécification OpenAPI d'abord	Écrire le code d'abord
Avantage	Contrat défini avant l'implémentation	spécification toujours synchronisée
Outillage	Prism (mock), génération de code	Swagger annotations
Idéal pour	APIs publiques, grandes équipes	Prototypes, petites équipes

Sécurité des API : API Keys, OAuth2, JWT et OpenID Connect

4.1 Les API Keys

Fonctionnement

Une API Key est une chaîne unique assignée à un client pour l'identifier. Simple à implémenter, elle est adaptée aux API machine-to-machine sans contexte utilisateur.

```
# API Key dans le header (recommandé)
GET /api/v1/books HTTP/1.1
X-API-Key: sk-live-abc123xyz456...

# API Key dans un query parameter (à éviter - reste dans les logs)
GET /api/v1/books?api_key=sk-live-abc123xyz456...
```

Avantages	Inconvénients
Simple à implémenter	Pas de contexte utilisateur
Pas de serveur d'autorisation nécessaire	Difficile à révoquer finement
Idéal pour les bots et scripts	Risque si exposée dans le code client

4.2 OAuth 2.0

Le problème que OAuth2 résout

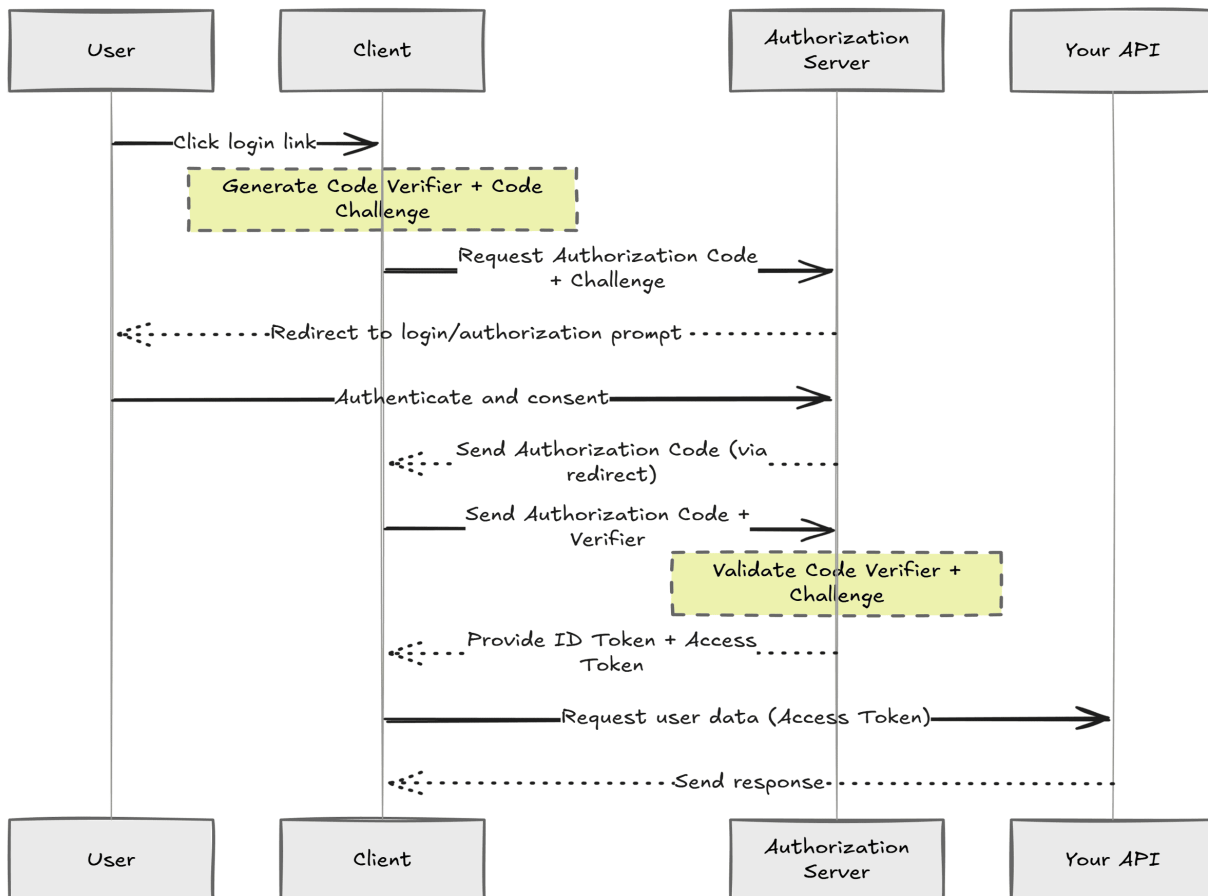
OAuth2 permet à un utilisateur d'autoriser une application tierce à accéder à ses ressources sans partager son mot de passe. Le standard est défini dans la RFC 6749.

Les 4 acteurs d'OAuth2
Resource Owner (RO) : l'utilisateur qui possède les données
Client : l'application qui veut accéder aux données
Authorization Server (AS) : délivre les tokens (ex: Keycloak , Auth0)
Resource Server (RS) : l'API qui protège les données

Les Grant Types principaux

Grant Type	Usage	Recommandé ?
Authorization Code + PKCE	Apps web/mobile avec utilisateur	Oui (standard)
Client Credentials	Machine-to-machine (pas d'utilisateur)	Oui
Device Code	TV, IoT, sans navigateur	Oui
Password (ROPC)	Migration legacy uniquement	Non (obsolète)
Implicit	Anciens SPAs	Non (obsolète)

Flux Authorization Code avec PKCE



1. App génère un `code_verifier` (aléatoire) + `code_challenge` (hash SHA256)

2. Redirection vers l'AS :

```
GET https://auth.exemple.fr/authorize
?response_type=code
&client_id=mon-app
&redirect_uri=https://app.fr/callback
&scope=read:books write:loans
&code_challenge=abc123...
&code_challenge_method=S256
```

3. Utilisateur se connecte et autorise → AS retourne un code

4. Échange du code contre un token :

```
POST https://auth.exemple.fr/token
{ grant_type: authorization_code, code: ..., code_verifier: ... }
```

5. AS retourne : { `access_token`, `refresh_token`, `expires_in` }

6. Appel à l'API protégée :

```
GET /api/v1/loans
Authorization: Bearer eyJhbGciOiJIJSUzIiNiJ9...
```

4.3 Les JSON Web Tokens (JWT)

Structure d'un JWT

Un JWT est composé de trois parties encodées en Base64URL et séparées par des points :
Header.**Payload**.**Signature**

```
# Header (algorithme de signature)
{ "alg": "RS256", "typ": "JWT" }

# Payload (claims)
{
  "sub": "user-123",           // Subject (ID utilisateur)
  "iss": "https://auth.fr",    // Issuer
  "aud": "api.bibliotheque.fr", // Audience
  "exp": 1735689600,           // Expiration (timestamp)
  "iat": 1735686000,           // Issued At
  "scope": "read:books write:loans",
  "roles": ["user", "admin"]
}

# Signature = RS256(base64(header) + '.' + base64(payload), clé_privée)
```

4.4 OpenID Connect (OIDC)

OpenID Connect est une couche d'identité construite au-dessus d'OAuth2. Là où **OAuth2** fournit l'**autorisation** ("que peut faire l'app ?"), **OIDC** fournit l'**authentification** ("qui est l'utilisateur ?").

OAuth2	OpenID Connect
Autorisation (accès aux ressources)	Authentification (identité de l'utilisateur)
Retourne un access_token	Retourne aussi un id_token
Scope: read:books, write:loans	Scope: openid, profile, email
"Cette app peut lire vos livres"	"Cet utilisateur est Jean Dupont"

4.5 Bonnes pratiques de sécurité API

- Toujours utiliser HTTPS — jamais HTTP en production
- Valider le JWT : signature, expiration, audience, issuer
- Utiliser des scopes fins : read:books plutôt que read:*
- Tokens à courte durée de vie (15min) + refresh tokens
- Stocker les tokens en mémoire, pas en localStorage
- Implémenter le rate limiting pour éviter les abus
- Logs et alertes sur les anomalies d'authentification

Patterns Microservices : API Gateway, Service Discovery et Circuit Breaker

5.1 Introduction aux Microservices

L'architecture microservices décompose une application en services indépendants, chacun responsable d'un domaine métier. Ces services communiquent via des APIs.

Architecture Monolithique	Architecture Microservices
Un seul déploiement	Déploiement indépendant par service
Scalabilité globale	Scalabilité ciblée par service
Simple à développer (début)	Complexité distribuée
Base de données partagée	Base de données par service
Failure = tout tombe	Failure isolée à un service

5.2 L'API Gateway

Rôle et responsabilités

L'API Gateway est le point d'entrée unique de l'architecture. Il agit comme proxy intelligent entre les clients et les microservices.

- Routage des requêtes vers le bon service
- Authentification et autorisation centralisées
- Rate limiting et quotas
- SSL termination
- Transformation de requêtes/réponses
- Agrégation de plusieurs services en une seule réponse
- Logging et monitoring centralisés

Exemple : AWS API Gateway / Nginx
Client → [API Gateway] → Service Livres
→ Service Utilisateurs
→ Service Emprunts
→ Service Notifications
Le client ne connaît que l'adresse de la Gateway.
Les services internes peuvent évoluer sans impacter le client.

Backend for Frontend (BFF)

Le pattern BFF consiste à créer une Gateway dédiée par type de client (web, mobile, TV), permettant d'adapter la réponse aux besoins spécifiques de chaque interface.

5.3 Le Service Discovery

Problématique

Dans un environnement dynamique (cloud, conteneurs), les services démarrent et s'arrêtent constamment. Leurs adresses IP changent. Comment un service A trouve-t-il l'adresse de service B ?

Client-Side vs Server-Side Discovery

Client-Side Discovery	Server-Side Discovery
Le client interroge le registre	Le load balancer interroge le registre
Ex: Netflix Eureka + Ribbon	Ex: AWS ELB, Kubernetes Service
Plus de flexibilité côté client	Client plus simple
Couplage au registre	Délégation au load balancer

```
# Exemple avec Consul (registre de services)

# Enregistrement d'un service
PUT http://consul:8500/v1/agent/service/register
{
  "Name": "service-livres",
  "Address": "10.0.1.5",
  "Port": 8080,
  "Check": { "HTTP": "http://10.0.1.5:8080/health", "Interval": "10s" }
}

# Découverte d'un service
GET http://consul:8500/v1/catalog/service/service-livres
```

5.4 Le Circuit Breaker

Le problème : cascade de pannes

Dans un système distribué, si un service est lent ou en panne, les appels s'accumulent, consomment des threads, et provoquent la panne des services qui en dépendent. C'est l'effet domino.

Les 3 états du Circuit Breaker

État	Description	Comportement
Closed (fermé)	Fonctionnement normal	Toutes les requêtes passent
Open (ouvert)	Trop d'erreurs détectées	Toutes les requêtes échouent rapidement (fail fast)
Half-Open (mi-ouvert)	Test de rétablissement	Quelques requêtes passent pour tester

```
// Exemple avec Resilience4j (Java/Node)
const circuitBreaker = CircuitBreaker.of('service-livres', {
  failureRateThreshold: 50,      // Ouvre si 50% d'erreurs
  waitDurationInOpenState: 10000, // 10s en état ouvert
  permittedNumberOfCallsInHalfOpenState: 3
});

const result = await circuitBreaker.executeAsync(() =>
  fetch('http://service-livres/api/books')
);
```

Autres patterns de résilience

Pattern	Description
Retry	Réessayer automatiquement en cas d'erreur transitoire (avec backoff exponentiel)
Timeout	Ne pas attendre indéfiniment — définir un délai maximum
Bulkhead	Isoler les ressources par service (pool de threads séparé)
Fallback	Réponse dégradée si le service est indisponible

Performance et Évolutivité : Caching, Rate Limiting et Webhooks

6.1 Le Caching

Les niveaux de cache

Niveau	Emplacement	Exemples
Client	Navigateur ou app mobile	HTTP Cache, Service Worker
Réseau	CDN, proxy reverse	Cloudflare, Varnish, Nginx
Serveur	Mémoire de l'application	Redis, Memcached
Base de données	Query cache	PostgreSQL cache, ORM cache

Les en-têtes HTTP de cache

```
# Réponse avec directives de cache
HTTP/1.1 200 OK
Cache-Control: public, max-age=3600, stale-while-revalidate=86400
ETag: "abc123"
Last-Modified: Mon, 13 Jan 2025 10:00:00 GMT

# Requête conditionnelle (304 si non modifié)
GET /api/v1/books/42
If-None-Match: "abc123"
If-Modified-Since: Mon, 13 Jan 2025 10:00:00 GMT

# Directives Cache-Control importantes
# public      → peut être mis en cache par CDN et navigateur
# private     → uniquement dans le navigateur (données personnelles)
# no-cache    → doit valider avec le serveur avant utilisation
# no-store    → jamais en cache (données sensibles)
# max-age=N   → durée en secondes
```

Stratégies de cache applicatif

Stratégie	Description	Usage
Cache-Aside	App gère le cache manuellement	Cas général — read heavy
Write-Through	Écriture en cache + BDD simultanément	Cohérence forte requise
Write-Back	Écriture en cache, sync BDD différée	Haute performance en écriture
Read-Through	Cache en proxy devant la BDD	Abstraction complète

6.2 Le Rate Limiting

Pourquoi limiter les taux ?

- Protéger l'API contre les abus et attaques DDoS
- Garantir l'équité entre les consommateurs

- Contrôler les coûts d'infrastructure
- Monétisation (tiers gratuit vs payant)

Algorithmes de Rate Limiting

Algorithme	Principe	Avantage
Fixed Window	N requêtes par fenêtre fixe (minute, heure)	Simple à implémenter
Sliding Window	N requêtes sur les N dernières secondes	Plus précis que fixed window
Token Bucket	Un bucket se remplit de tokens à taux constant	Tolère les bursts
Leaky Bucket	File d'attente à débit constant	Sortie régulière garantie

Headers de Rate Limiting

```

HTTP/1.1 200 OK
X-RateLimit-Limit: 100           # Quota total
X-RateLimit-Remaining: 23       # Requetes restantes
X-RateLimit-Reset: 1735689600   # Timestamp de reset
Retry-After: 45                 # Secondes avant retry (sur 429)

# En cas de dépassement
HTTP/1.1 429 Too Many Requests
{
  "error": "rate_limit_exceeded",
  "message": "100 requêtes par heure autorisées. Retry après 45 secondes.",
  "retryAfter": 45
}

```

6.3 Les Webhooks

Pull vs Push

Polling (Pull)	Webhooks (Push)
Client interroge régulièrement l'API	Serveur notifie le client lors d'un événement
"Est-ce que mon emprunt est validé ?" × 1000	"Ton emprunt vient d'être validé !"
Gaspillage de ressources	Efficace et temps-réel
Implémentation simple côté client	Nécessite un endpoint côté client

Implémenter un Webhook

```

// 1. Le client enregistre son URL de callback
POST /api/v1/webhooks
{
  "url": "https://app.client.fr/hooks/bibliotheque",
  "events": ["loan.created", "loan.returned", "book.available"],
  "secret": "whsec_abc123"
}

```

```
// 2. Lors d'un événement, le serveur envoie une requête POST
POST https://app.client.fr/hooks/bibliotheque
X-Signature-256: sha256=abc123... // Sécurité : vérifier ce header
{
  "event": "loan.created",
  "timestamp": "2025-01-13T10:30:00Z",
  "data": { "loanId": 789, "userId": 42, "bookId": 15 }
}

// 3. Le client répond 200 OK pour acquitter
// → Si pas de 200, le serveur re-tentera (retry avec backoff)
```

Sécuriser les Webhooks

- Signer les payloads avec HMAC-SHA256 et un secret partagé
- Vérifier la signature côté client avant de traiter
- Valider le timestamp pour éviter les replay attacks
- Utiliser HTTPS uniquement
- Implémenter l'idempotence (même event peut être livré plusieurs fois)

6.4 Server-Sent Events et WebSockets

Technologie	Direction	Usage
REST + Polling	Client → Serveur (répété)	Cas simples, pas de temps réel
Webhooks	Serveur → Client (événements)	Notifications asynchrones
SSE	Serveur → Client (flux)	Notifications en temps réel (simplex)
WebSocket	Bidirectionnel	Chat, jeux, collaboration live

FIN DU COURS